```
    F1dim(VecDoub_I &pp, VecDoub_I &xii, T &funcc) : p(pp),
        xi(xii), n(pp.size()), func(funcc), xt(n) {}
    Constructor takes as inputs an n-dimensional point p[0..n-1] and an n-dimensional di-
    rection xi[0..n-1] from linmin, as well as the function or functor that takes a vector
    argument.
    Doub operator() (const Doub x)
    Functor returning value of the given function along a one-dimensional line.
    {
        for (Int j=0;j<n;j++)
            xt[j]=p[j]+x*xi[j];
        return func(xt);
    }
};
```

10.7 Direction Set (Powell's) Methods in Multidimensions

With a routine for line minimization in hand, you might think of this simple method for general multidimensional minimization: Take the unit vectors $\mathbf{e}_0, \mathbf{e}_1, \ldots \mathbf{e}_{N-1}$ as a *set of directions*. Using `linmin`, move along the first direction to its minimum, then *from there* along the second direction to *its* minimum, and so on, cycling through the whole set of directions as many times as necessary, until the function stops decreasing.

This simple method is actually not too bad for many functions. Even more interesting is why it *is* bad, i.e., very inefficient, for some other functions. Consider a function of two dimensions whose contour map (level lines) happens to define a long, narrow valley at some angle to the coordinate basis vectors (see Figure 10.7.1). Then the only way "down the length of the valley" going along the basis vectors at each stage is by a series of many tiny steps. More generally, in N dimensions, if the function's second derivatives are much larger in magnitude in some directions than in others, then many cycles through all N basis vectors will be required in order to get anywhere. This condition is not all that unusual; according to Murphy's Law, you should count on it.

Obviously what we need is a better set of directions than the $\mathbf{e}_i$'s. All *direction set methods* consist of prescriptions for updating the set of directions as the method proceeds, attempting to come up with a set that either (i) includes some very good directions that will take us far along narrow valleys, or else (more subtly) (ii) includes some number of "noninterfering" directions with the special property that minimization along one is not "spoiled" by subsequent minimization along another, so that interminable cycling through the set of directions can be avoided.

10.7.1 Conjugate Directions

This concept of "noninterfering" directions, more conventionally called *conjugate directions*, is worth making mathematically explicit.

First, note that if we minimize a function along some direction $\mathbf{u}$, then the gradient of the function must be perpendicular to $\mathbf{u}$ at the line minimum; if not, then there would still be a nonzero directional derivative along $\mathbf{u}$.

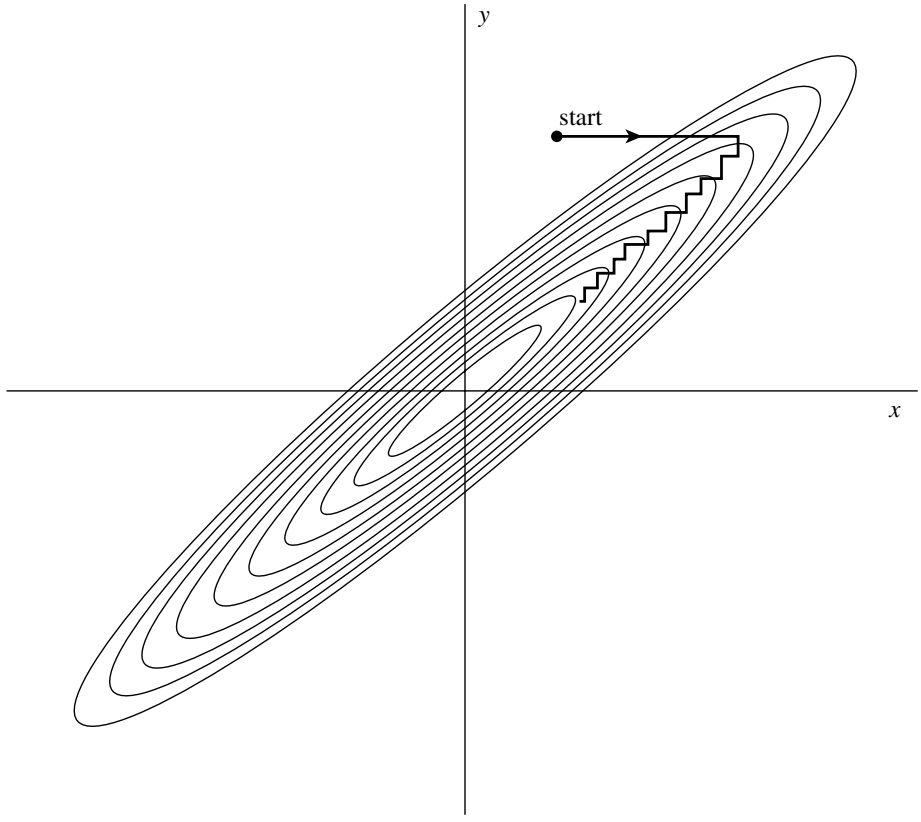

Figure 10.7.1. Successive minimizations along coordinate directions in a long, narrow "valley" (shown as contour lines). Unless the valley is optimally oriented, this method is extremely inefficient, taking many tiny steps to get to the minimum, crossing and re-crossing the principal axis.

Next take some particular point $\mathbf{P}$ as the origin of the coordinate system with coordinates $\mathbf{x}$. Then any function f can be approximated by its Taylor series

$$\begin{aligned} f(\mathbf{x}) &= f(\mathbf{P}) + \sum_i \frac{\partial f}{\partial x_i} x_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} x_i x_j + \cdots \\ &\approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} \end{aligned} \tag{10.7.1}$$

where

$$c \equiv f(\mathbf{P}) \qquad \mathbf{b} \equiv -\nabla f|_{\mathbf{P}} \qquad [\mathbf{A}]_{ij} \equiv \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\mathbf{P}} \tag{10.7.2}$$

The matrix $\mathbf{A}$ whose components are the second partial derivative matrix of the function is called the *Hessian matrix* of the function at $\mathbf{P}$.

In the approximation of (10.7.1), the gradient of f is easily calculated as

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \tag{10.7.3}$$

(This implies that the gradient will vanish — the function will be at an extremum — at a value of $\mathbf{x}$ obtained by solving $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. We will return to this idea in §10.9!)

How does the gradient ∇f *change* as we move along some direction? Evidently

$$\delta(\nabla f) = \mathbf{A} \cdot (\delta \mathbf{x}) \tag{10.7.4}$$

Suppose that we have moved along some direction $\mathbf{u}$ to a minimum and now propose to move along some new direction $\mathbf{v}$. The condition that motion along $\mathbf{v}$ not *spoil* our minimization along $\mathbf{u}$ is just that the gradient stay perpendicular to $\mathbf{u}$, i.e., that the change in the gradient be perpendicular to $\mathbf{u}$. By equation (10.7.4) this is just

$$0 = \mathbf{u} \cdot \delta(\nabla f) = \mathbf{u} \cdot \mathbf{A} \cdot \mathbf{v} \tag{10.7.5}$$

When (10.7.5) holds for two vectors $\mathbf{u}$ and $\mathbf{v}$, they are said to be *conjugate*. When the relation holds pairwise for all members of a set of vectors, they are said to be a conjugate set. If you do successive line minimizations of a function along a conjugate set of directions, then you don't need to redo any of those directions (unless, of course, you spoil things by minimizing along a direction that they are *not* conjugate to).

A triumph for a direction set method is to come up with a set of N linearly independent, mutually conjugate directions. Then, one pass of N line minimizations will put it exactly at the minimum of a quadratic form like (10.7.1). For functions f that are not exactly quadratic forms, it won't be exactly at the minimum, but repeated cycles of N line minimizations will in due course converge *quadratically* to the minimum.

10.7.2 Powell's Quadratically Convergent Method

Powell first discovered a direction set method that does produce N mutually conjugate directions. Here is how it goes: Initialize the set of directions $\mathbf{u}_i$ to the basis vectors,

$$\mathbf{u}_i = \mathbf{e}_i \qquad i = 0, \ldots, N-1 \tag{10.7.6}$$

Now repeat the following sequence of steps ("basic procedure") until your function stops decreasing:

- Save your starting position as $\mathbf{P}_0$.
- For $i = 0, \ldots, N-1$, move $\mathbf{P}_i$ to the minimum along direction $\mathbf{u}_i$ and call this point $\mathbf{P}_{i+1}$.
- For $i = 0, \ldots, N-2$, set $\mathbf{u}_i \leftarrow \mathbf{u}_{i+1}$.
- Set $\mathbf{u}_{N-1} \leftarrow \mathbf{P}_N - \mathbf{P}_0$.
- Move $\mathbf{P}_N$ to the minimum along direction $\mathbf{u}_{N-1}$ and call this point $\mathbf{P}_0$.

Powell, in 1964, showed that, for a quadratic form like (10.7.1), k iterations of the above basic procedure produce a set of directions $\mathbf{u}_i$ whose last k members are mutually conjugate. Therefore, N iterations of the basic procedure, amounting to $N(N+1)$ line minimizations in all, will exactly minimize a quadratic form. Brent [1] gives proofs of these statements in accessible form.

Unfortunately, there is a problem with Powell's quadratically convergent algorithm. The procedure of throwing away, at each stage, $\mathbf{u}_0$ in favor of $\mathbf{P}_N - \mathbf{P}_0$ tends to produce sets of directions that "fold up on each other" and become linearly dependent. Once this happens, the procedure finds the minimum of the function f only over a subspace of the full N-dimensional case; in other words, it gives the wrong answer. Therefore, the algorithm must not be used in the form given above.

There are a number of ways to fix up the problem of linear dependence in Powell's algorithm, among them:

1. You can reinitialize the set of directions $\mathbf{u}_i$ to the basis vectors $\mathbf{e}_i$ after every N or $N+1$ iterations of the basic procedure. This produces a serviceable method, which we commend to you if quadratic convergence is important for your application (i.e., if your functions are close to quadratic forms and if you desire high accuracy).

2. Brent points out that the set of directions can equally well be reset to the columns of any orthogonal matrix. Rather than throw away the information on conjugate directions already built up, he resets the direction set to calculated principal directions of the matrix $\mathbf{A}$ (which he gives a procedure for determining). The calculation is essentially a singular value decomposition algorithm (see §2.6). Brent has a number of other cute tricks up his sleeve, and his modification of Powell's method is probably the best presently known. Consult [1] for a detailed description and listing of the program. Unfortunately it is rather too elaborate for us to include here.

3. You can give up the property of quadratic convergence in favor of a more heuristic scheme (due to Powell) that tries to find a few good directions along narrow valleys instead of N necessarily conjugate directions. This is the method that we now implement. (It is also the version of Powell's method given in Acton [2], from which parts of the following discussion are drawn.)

10.7.3 Discarding the Direction of Largest Decrease

The fox and the grapes: Now that we are going to give up the property of quadratic convergence, was it so important after all? That depends on the function that you are minimizing. Some applications produce functions with long, twisty valleys. Quadratic convergence is of no particular advantage to a program that must slalom down the length of a valley floor that twists one way and another (and another, and another, ... — there are N dimensions!). Along the long direction, a quadratically convergent method is trying to extrapolate to the minimum of a parabola that just isn't (yet) there while the conjugacy of the $N-1$ transverse directions keeps getting spoiled by the twists.

Sooner or later, however, we do arrive at an approximately ellipsoidal minimum (cf. equation 10.7.1 when $\mathbf{b}$, the gradient, is zero). Then, depending on how much accuracy we require, a method with quadratic convergence can save us several times N^2 extra line minimizations, since quadratic convergence *doubles* the number of significant figures at each iteration.

The basic idea of our now-modified Powell's method is still to take $\mathbf{P}_N - \mathbf{P}_0$ as a new direction; it is, after all, the average direction moved in after trying all N possible directions. For a valley whose long direction is twisting slowly, this direction is likely to give us a good run along the new long direction. The change is to discard the old direction along which the function f made its *largest decrease*. This seems paradoxical, since that direction was the *best* of the previous iteration. However, it is also likely to be a major component of the new direction that we are adding, so dropping it gives us the best chance of avoiding a buildup of linear dependence.

There are a couple of exceptions to this basic idea. Sometimes it is better *not* to add a new direction at all. Define

$$f_0 \equiv f(\mathbf{P}_0) \qquad f_N \equiv f(\mathbf{P}_N) \qquad f_E \equiv f(2\mathbf{P}_N - \mathbf{P}_0) \tag{10.7.7}$$

Here f_E is the function value at an "extrapolated" point somewhat further along the proposed new direction. Also define Δf to be the magnitude of the largest decrease along one particular direction of the present basic procedure iteration. (Δf is a positive number.) Then:

1. If $f_E \geq f_0$, then keep the old set of directions for the next basic procedure, because the average direction $\mathbf{P}_N - \mathbf{P}_0$ is all played out.

2. If $2\,(f_0 - 2f_N + f_E)\,[(f_0 - f_N) - \Delta f]^2 \geq (f_0 - f_E)^2 \Delta f$, then keep the old set of directions for the next basic procedure, because either (i) the decrease along the average direction was not primarily due to any single direction's decrease, or (ii) there is a substantial second derivative along the average direction and we seem to be near to the bottom of its minimum.

The following routine implements Powell's method in the version just described. In the routine, `xi` is the matrix whose columns are the set of directions $\mathbf{n}_i$; otherwise the correspondence of notation should be self-evident. If the function to be minimized is provided as a functor `Func`

```
struct Func {
    Doub operator()(VecDoub_I &x);
};
```

then the normal calling sequence for `Powell` looks something like this:

```
VecDoub p = ...;
Func func;
Powell<Func> powell(func);
p=powell.minimize(p);          OK to overwrite initial guess.
```

The function value at the minimum is available as `powell.fret`.

If, on the other hand, the function to be minimized is provided as a normal C++ function,

```
Doub func(VecDoub_I &x);
```

then the constructor call looks like this instead:

```
Powell<Doub (VecDoub_I &)> powell(func);
```

Note that the constructor takes an optional argument that specifies the function tolerance for the minimization.

mins_ndim.h

```
template <class T>
struct Powell : Linemethod<T> {
```

Multidimensional minimization by Powell's method.

```
    Int iter;
    Doub fret;                      Value of the function at the minimum.
    using Linemethod<T>::func;      Variables from a templated base class are not auto-
    using Linemethod<T>::linmin;        matically inherited.
    using Linemethod<T>::p;
    using Linemethod<T>::xi;
    const Doub ftol;
    Powell(T &func, const Doub ftoll=3.0e-8) : Linemethod<T>(func),
        ftol(ftoll) {}
```

Constructor arguments are `func`, the function or functor to be minimized, and an optional argument `ftoll`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness.

```
    VecDoub minimize(VecDoub_I &pp)
```

Minimization of a function or functor `n` variables. Input consists of an initial starting point `pp[0..n-1]`. The initial matrix `ximat[0..n-1][0..n-1]`, whose columns contain the initial set of directions, is set to the `n` unit vectors. Returned is the best point found, at which point `fret` is the minimum function value and `iter` is the number of iterations taken.

```
    {
        Int n=pp.size();
        MatDoub ximat(n,n,0.0);
        for (Int i=0;i<n;i++) ximat[i][i]=1.0;
        return minimize(pp,ximat);
    }
    VecDoub minimize(VecDoub_I &pp, MatDoub_IO &ximat)
    Alternative interface: Input consists of the initial starting point pp[0..n-1] and an initial
    matrix ximat[0..n-1][0..n-1], whose columns contain the initial set of directions. On
    output ximat is the then-current direction set.
    {
        const Int ITMAX=200;          Maximum allowed iterations.
        const Doub TINY=1.0e-25;      A small number.
        Doub fptt;
        Int n=pp.size();
        p=pp;
        VecDoub pt(n),ptt(n);
        xi.resize(n);
        fret=func(p);
        for (Int j=0;j<n;j++) pt[j]=p[j];  Save the initial point.
        for (iter=0;;++iter) {
            Doub fp=fret;
            Int ibig=0;
            Doub del=0.0;             Will be the biggest function decrease.
            for (Int i=0;i<n;i++) {   In each iteration, loop over all directions in the set.
                for (Int j=0;j<n;j++) xi[j]=ximat[j][i];     Copy the direction,
                fptt=fret;
                fret=linmin();                  minimize along it,
                if (fptt-fret > del) {          and record it if it is the largest decrease
                    del=fptt-fret;                  so far.
                    ibig=i+1;
                }
            }                                   Here comes the termination criterion:
            if (2.0*(fp-fret) <= ftol*(abs(fp)+abs(fret))+TINY) {
                return p;
            }
            if (iter == ITMAX) throw("powell exceeding maximum iterations.");
            for (Int j=0;j<n;j++) {             Construct the extrapolated point and the
                ptt[j]=2.0*p[j]-pt[j];              average direction moved. Save the
                xi[j]=p[j]-pt[j];                   old starting point.
                pt[j]=p[j];
            }
            fptt=func(ptt);                     Function value at extrapolated point.
            if (fptt < fp) {
                Doub t=2.0*(fp-2.0*fret+fptt)*SQR(fp-fret-del)-del*SQR(fp-fptt);
                if (t < 0.0) {
                    fret=linmin();              Move to the minimum of the new direc-
                    for (Int j=0;j<n;j++) {         tion, and save the new direction.
                        ximat[j][ibig-1]=ximat[j][n-1];
                        ximat[j][n-1]=xi[j];
                    }
                }
            }
        }
    }
};
```

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 7.[1]